函数栈调用逻辑（通俗+完整版）

函数栈的调用逻辑（通俗+完整版）

你可以把**函数调用栈（Call Stack）**理解成：

程序专门用来**记录“函数跑到哪了”**的一个**后进先出（LIFO）**的记事本。

每调用一次函数，就往本子上**压一页**；

函数执行完返回，就把这一页**撕掉**。

1. 栈的基本规则

- 内存区域：**栈区（stack）**
- 特点：**先进后出**

先调用的函数后返回，后调用的函数先返回。

2. 一次普通函数调用的完整过程

以这段代码为例：

Code block

```
void b() { /* ... */ }
void a() { b(); }
int main() { a(); }
```

调用流程：

1. 程序先跑 `main`
2. 遇到 `a()`：
   - 记住**返回地址**：等a执行完要回到main的哪一行
   - 给 `a` 分配栈帧（存放参数、局部变量）
   - 跳去执行 `a`
3. `a` 里遇到 `b()`：
   - 同样记住返回地址

- 给 `b` 分配栈帧
- 跳去执行 `b`

4. `b` 执行完：
   - 销毁 `b` 的栈帧
   - 按记录的地址回到 `a`
5. `a` 执行完：
   - 销毁 `a` 的栈帧
   - 回到 `main`
6. `main` 结束，程序退出

栈的变化：

`main` → 压入 `a` → 压入 `b` → 弹出 `b` → 弹出 `a`

3. 递归时的栈调用逻辑（最关键）

拿递归求阶乘 `factorial(3)` 举例：

Code block

```
int fact(int n) {
    if(n==1) return 1;
    return n * fact(n-1);
}
```

栈的完整变化（压栈 → 递）

1. main 调用 fact(3)

栈：[fact(3)]

2. fact(3) 调用 fact(2)

栈：[fact(3), fact(2)]

3. fact(2) 调用 fact(1)

栈：[fact(3), fact(2), fact(1)]

到这里，**递归“递”的过程结束**，栈达到最深。

回溯返回（弹栈 → 归）

4. fact(1) 满足终止条件，返回 1

弹出 fact(1)，栈：[fact(3), fact(2)]

5. fact(2) 计算：2 * 1 = 2，返回

弹出 fact(2)，栈：[fact(3)]

6. fact(3) 计算：3 * 2 = 6，返回

弹出 fact(3)，栈空

4. 栈帧里到底存了什么？

每一层函数，系统都会自动保存：

- 函数的**返回地址**（回到哪）
- 传入的**参数**
- 函数内部的**局部变量**
- 上一层栈的位置信息

这些合起来叫一个**栈帧（Stack Frame）**。

5. 为什么递归太深会崩溃？

因为**栈空间很小**（通常几MB）。

如果你写：

Code block

```
// 没有终止条件
void f() { f(); }
```

函数会**无限压栈**，栈空间被撑爆，就是 **栈溢出（Stack Overflow）**。

一句话总结

- 函数调用 = **压栈**
- 函数返回 = **弹栈**
- 递归 = 反复压栈，直到终止条件，再依次弹栈
- 栈是**先进后出**，所以递归才有“先递进去、再归回来”的顺序

这个逻辑搞懂，递归、函数调用、崩溃原因你就全通了。

（注：文档部分内容可能由 AI 生成）